

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1

SECTION A – SCENARIO BASED ASSESSMENT

Q23. A hospital system stores emergency patient data unsorted for quick updates. Doctors need to locate records rapidly.

- Explain how Linear Search helps in this scenario.
- Analyze its performance in different cases.
- Justify why sorting may not always be preferred.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30%	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20%	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15%	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10%	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%				
Application of Concepts	30%				
Analytical & Decision-Making Skills	20%				
Solution Design & Approach	15%				
Clarity, Justification & Presentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q23. Application: Hospital Emergency Patient Data Using Linear Search

Introduction

A hospital emergency department handles a large number of patients every day. New patient records are added continuously, and existing records are updated frequently. Since emergency treatment is the highest priority, records are often stored **unsorted** to allow quick insertion and updating. Doctors and hospital staff need to search for patient information such as **Patient ID, Name, Blood Group, or Contact Number** as quickly as possible. In this situation, **Linear Search** is a suitable searching technique because it can search directly through unsorted data without requiring any additional preparation.

a) Explain how Linear Search helps in this scenario

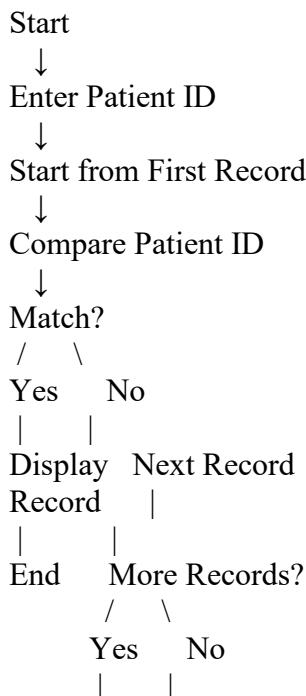
Definition

Linear Search is a searching technique in which each record is checked one by one from the beginning until the required record is found or the end of the list is reached.

Algorithm

- Start from the first patient record.
- Compare the required Patient ID or Name with the current record.
- If both match, display the patient's information.
- If not matched, move to the next record.
- Repeat until the record is found or all records are checked.
- If no match is found, display "**Patient Record Not Found.**"

Flow of Linear Search



Continue Record Not Found

|
End

Example

Patient Records

Position	Patient ID	Patient Name
----------	------------	--------------

1	101	Ramesh
---	-----	--------

2	105	Suresh
---	-----	--------

3	110	Priya
---	-----	-------

4	115	Anil
---	-----	------

5	120	Divya
---	-----	-------

Search for Patient ID = 115

Comparison 1:

$101 \neq 115$

↓

Comparison 2:

$105 \neq 115$

↓

Comparison 3:

$110 \neq 115$

↓

Comparison 4:

$115 = 115$

↓

Patient Record Found

Why Linear Search is Suitable

Patient records remain unsorted.

New patients can be added immediately.

No sorting process is required.

Easy to implement.

Suitable for emergency situations where quick updates are important.

Works even when patient records change frequently.

b) Analyze its performance in different cases

The efficiency of Linear Search depends on the position of the required patient record.

1. Best Case

The required patient record is found in the first position.

Example

Searching Patient ID **101**

Only **one comparison** is needed.

Time Complexity

$O(1)$

Explanation

This is the fastest situation because the first record itself matches the search value.

2. Average Case

The patient record is somewhere in the middle of the list.

Example

Searching Patient ID **110**

Approximately half of the records are checked.

Time Complexity

$O(n)$

Explanation

On average, the algorithm checks about **$n/2$** records before finding the patient

3. Worst Case

The patient record is at the last position or does not exist.

Example

Searching Patient ID **130**

All records are checked before concluding that the patient is not available.

Time Complexity

$O(n)$

Explanation

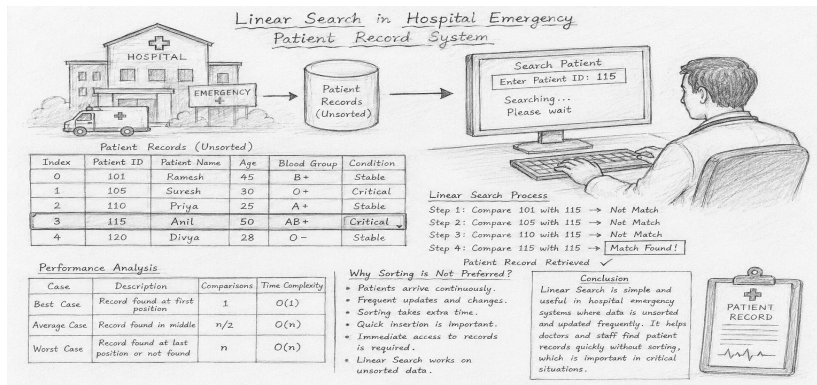
The search examines every record one by one.

Time Complexity Table

Case	Number of Comparisons	Time Complexity
Best Case	1	$O(1)$
Average Case	$n/2$	$O(n)$
Worst Case	n	$O(n)$

Space Complexity

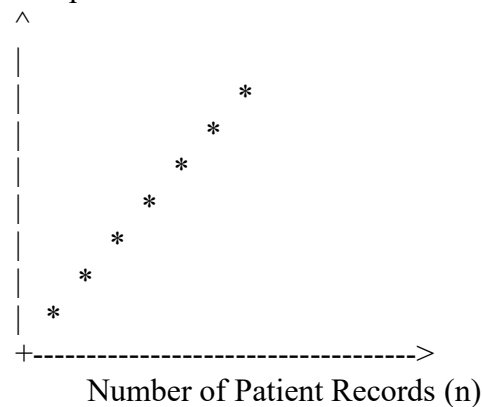
Linear Search uses only one extra variable during searching.



Space Complexity = $O(1)$

Performance Graph

Comparisons



Linear Search = $O(n)$

The graph shows that as the number of patient records increases, the number of comparisons also increases.

c) Justify why sorting may not always be preferred

Although sorting helps in faster searching methods such as Binary Search, it is not always suitable in hospital emergency systems.

Reasons

1. Continuous Arrival of Patients

Emergency patients arrive every minute.

Sorting after each new entry wastes valuable time.

2. Frequent Updates

Patient details such as:

Bed Number

Doctor Assigned

Medical Status

Emergency Level

change frequently.

Maintaining sorted order becomes difficult.

3. Faster Data Entry

Hospital staff should quickly register new patients.

Unsorted storage allows immediate insertion.

4. Time-Critical Environment

Doctors cannot wait for records to be sorted before treatment.

Immediate searching is more important.

5. Extra Processing Cost

Sorting large numbers of records repeatedly consumes additional CPU time and resources.

6. Simplicity

Linear Search works directly on unsorted records.

No preprocessing is required.

Comparison: Sorted vs Unsorted Data

Feature	Unsorted Data + Linear Search	Sorted Data + Binary Search
Sorting Required	No	Yes
Easy to Insert New Records	Yes	No

Feature	Unsorted Data + Linear Search	Sorted Data + Binary Search
Frequent Updates	Easy	Difficult
Suitable for Emergency System	Yes	No
Search Speed	$O(n)$	$O(\log n)$

Advantages of Linear Search

Simple algorithm.

Easy to understand and implement.

Works on sorted and unsorted data.

No preprocessing required.

Requires very little memory.

Suitable for small datasets.

Best for frequently changing data.

Easy to debug.

Supports dynamic data insertion.

Disadvantages of Linear Search

Slow for very large datasets.

Checks records one by one.

More comparisons compared to Binary Search.

Performance decreases as the dataset grows.

Not suitable for searching millions of records.

Applications of Linear Search

Hospital patient record management.

Student database search.

Employee record management.

Library management systems.

Product inventory search.

Contact list search.

Attendance systems.

Small business databases.

Real-Life Example

A patient arrives at the emergency ward with Patient ID **115**. Since the hospital database is unsorted, the computer starts checking records one by one until Patient ID **115** is found. The patient's previous medical history is displayed immediately, allowing doctors to begin treatment without delay.

EXECUTION/ OUTPUT

```
===== HOSPITAL EMERGENCY PATIENT RECORD SYSTEM : LINEAR SEARCH =====
```

```
Patient Records (Unsorted):
```

Index	Patient ID	Name	Age	Condition	Blood Group
0	101	Ramesh	45	Stable	B+
1	108	Suresh	30	Critical	O+
2	115	Priya	25	Stable	A+
3	123	Anil	50	Critical	AB+
4	130	Divya	28	Stable	O-
5	135	Kiran	40	Stable	B-

```
Enter Patient ID to search: 123
```

```
Searching for Patient ID = 123 using Linear Search...
```

```
Step 1: Comparing with index 0 (Patient ID = 101) -> Not Match
Step 2: Comparing with index 1 (Patient ID = 108) -> Not Match
Step 3: Comparing with index 2 (Patient ID = 115) -> Not Match
Step 4: Comparing with index 3 (Patient ID = 123) -> Match Found!
```

```
Patient Record Found Successfully!
```

```
-----
Patient ID : 123
Name       : Anil
Age        : 50
Condition  : Critical
Blood Group : AB+
-----
```

```
Total Comparisons: 4
```

```
Record found successfully.
```

Conclusion

In a hospital emergency system, patient records are updated continuously, making it impractical to keep the data sorted. **Linear Search** provides a simple and reliable way to locate patient records directly from an unsorted database. It requires **O(1)** time in the best case and **O(n)** time in the average and worst cases, while using only **O(1)** extra memory. Although it is slower than Binary Search for very large datasets, its ability to work efficiently with unsorted and frequently changing data makes it the most practical choice for emergency hospital record management.

